

GUI interface in Python

- Graphical User Interfaces (GUIs) in Python allow users to interact with applications through visual elements like buttons, menus, windows, and text boxes, rather than solely through command-line interfaces.

Popular Python GUI Frameworks:

- **Tkinter:**
- Python's standard and built-in GUI library.
- Provides a lightweight wrapper around the Tcl/Tk toolkit.
- Cross-platform, ensuring applications function similarly across Windows, macOS, and Linux.
- Suitable for developing simple to moderately complex desktop applications and learning GUI programming basics.

- **PyQt:**
- A set of Python bindings for the Qt application framework.
- Offers a comprehensive suite of widgets and features for building sophisticated, high-performance applications.
- Supports cross-platform development, including desktop and mobile environments.
- **wxPython:**
- A wrapper for the wxWidgets C++ library.
- Provides a native look and feel across different operating systems.
- Known for its extensive set of widgets and robust features for desktop application development.

- **Kivy:**
- Designed for creating modern and responsive user interfaces, particularly for multi-touch applications and mobile devices.
- Employs a declarative language (KV language) for UI design, enhancing development speed.

Key Concepts in Python GUI Development:

- **Widgets:**
 - The fundamental building blocks of a GUI, such as buttons, labels, entry fields, and scrollbars.
- **Layout Management:**
 - Techniques for arranging widgets within a window, including packing, placing, and gridding.
- **Event Handling:**
 - Mechanisms for responding to user interactions, such as button clicks, key presses, or mouse movements, by binding them to specific functions.

Tkinter

What is Tkinter?

- Tkinter is the standard GUI (Graphical User Interface) library that comes built-in with Python.
- It lets you build desktop apps with windows, buttons, labels, input boxes, and more.
 - *Comes pre-installed with Python (no extra downloads in most cases)
 - *Used to create windows, forms, dialogs, and interactive interfaces
 - *Beginner-friendly and great for quick prototypes.

- What Can You Build With Tkinter?
- You can build:
 - * Login/registration forms
 - * Chatbots and messengers
 - * Calculators and converters
 - * File explorers and text editors
 - * Dashboard UIs

Basic Tkinter Concepts

Here are some ****key components**** you'll use in Tkinter:

Widget	Description
Tk()	The main app window (root window)
Label	Shows text or images
Entry	Input field for text
Button	Clickable button
Text	Multi-line text box
Frame	Container to group other widgets
Canvas	For drawing shapes/graphics
messagebox	Pop-up dialog box

Minimal Tkinter Example

- Python

```
import tkinter as tk
```

```
root = tk.Tk()  
root.title("My First App")  
root.geometry("300x200")
```

```
label = tk.Label(root, text="Hello, Tkinter!")  
label.pack(pady=20)
```

```
entry = tk.Entry(root)  
entry.pack()
```

```
def on_click():  
    name = entry.get()  
    label.config(text=f"Hi, {name}!")
```

```
button = tk.Button(root, text="Say Hello", command=on_click)  
button.pack(pady=10)
```

```
root.mainloop()
```

Step 1 — Import Tkinter and Create a Window

```
import tkinter as tk  
from tkinter import messagebox  
# Create main window  
window = tk.Tk()  
window.title("Login Page")  
window.geometry("300x200")
```

- **Step 2 — Add Labels and Entry Fields**

Username label and text entry box

```
tk.Label(window, text="Username").pack(pady=5)
```

```
username_entry = tk.Entry(window)
```

```
username_entry.pack()
```

Password label and text entry box

```
tk.Label(window, text="Password").pack(pady=5)
```

```
password_entry = tk.Entry(window, show="*")
```

show * instead of real password

```
password_entry.pack()
```

- **Step 3 — Add Login Button and Logic**

```
def login():  
    username = username_entry.get()  
    password = password_entry.get()  
    # Example: hardcoded credentials  
    if username == "admin" and password == "1234":  
        messagebox.showinfo("Login", "Login Successful!")  
    else:  
        messagebox.showerror("Login", "Invalid username or  
            password")  
    login_button = tk.Button(window, text="Login",  
        command=login)  
    login_button.pack(pady=10)
```

- **Step 4 — Run the App**

`window.mainloop()`

- Full Code Example

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
# Create window
```

```
window = tk.Tk()window.title("Login Page")
```

```
window.geometry("300x200")
```

```
# Username label and text entry
tk.Label(window, text="Username").pack(pady=5)
username_entry = tk.Entry(window)
username_entry.pack()
```

```
# Password label and text entry
tk.Label(window, text="Password").pack(pady=5)
password_entry = tk.Entry(window, show="*")
password_entry.pack()
```

```
# Login button
def login():
    username = username_entry.get()
    password = password_entry.get()
    if username == "admin" and password == "1234":
        messagebox.showinfo("Login", "Login Successful!")
    else:
        messagebox.showerror("Login", "Invalid username or password")
tk.Button(window, text="Login", command=login).pack(pady=10)
window.mainloop()
```

- **Optional — Add “Clear” and “Exit” Buttons**

```
def clear():
```

```
    username_entry.delete(0, tk.END)
```

```
    password_entry.delete(0, tk.END)
```

```
def close():    window.destroy()
```

```
tk.Button(window, text="Clear",  
          command=clear).pack()
```

```
tk.Button(window, text="Exit",  
          command=close).pack()
```


- **Step 5 (Optional) — Store Users in Database (SQLite)**
- If you want a **real login system** instead of hardcoded credentials, you can:
- Create a users.db file using sqlite3.
- Save username/passwords (hashed) in it.
- Validate during login.